DoubleVerify

DS Classification Assignment

Overview

This assignment simulates a real-world classification problem. Given a set of images, **your goal is to build a classifier and detect images that contain superimposed text related to alcoholic beverages.**
Your task is to explore the data, analyze it, and build a classification pipeline that achieves the best metrics you can.
After submission, your classifier's performance may be evaluated on an unseen set.

Background

Doubleverify's *Universal Content Intelligence* is an online classification engine that powers expansive content categorization. This engine analyzes key types of content, including visual, audio, text, and others, across all channels, devices, and formats.
In this task, you will build a small component of a toy classification engine focusing on text extraction (OCR) and text-based classification for alcohol references in images.

Task

The assignment tests your ability to justify choices and derive insights, so make sure to document your reasoning.

1. Explore the dataset and understand the inputs and required outputs.
2. Design a classification pipeline and explain the architecture you chose.
3. Suggest appropriate metrics (intermediate and/or final) that will help you assess the performance of your pipeline.
4. Implement your choice of a pipeline, using the provided data.
5. Report your results and other findings that you feel valuable to share.

©2020 DoubleVerify. All rights reserved.

Examples of images from the data set:

Image	text	label
	Don’t miss limited edition nightlife bourbon event	alcohol
	energy drink	Non-alcohol

Data & Resources

1. The Dataset:
    - A set of images containing text. The images are split into two folders (“Alcohol” and “Non-alcohol”) based on whether the text contains mentions of alcohol or not.
2. OCR Packages
    - In order to extract text from images, you can use an off-the-shelf Python library for OCR (OCR = Optical Character Recognition), both of the following libraries are easy to set up and do not require a GPU to run but you may use any other reproducible method/library you prefer. See Installation and usage guides in the provided links. In addition, basic examples can be found at the end of this document.
        - EasyOCR
        - PaddleOCR
3. Hardware Constraints: CPU-only execution (no GPU) to keep it simple

©2020 DoubleVerify. All rights reserved.

What We're Looking For

- **Robustness:** How well does your pipeline handle noisy, distorted, or adversarial text?
- **Explainability:** Document why you chose certain approaches or parameters.
- **Performance:** We'll measure your final classifier on unseen data.
- **Creativity & Initiative:** Are you introducing any custom text-cleaning or advanced post-processing steps? Are you using domain knowledge about alcohol references or brand synonyms?

Deliverables

1. Python Code
    - Well-documented including requirements.txt.
    - Must reproduce consistent results in a clean environment.
    - A stand-alone inference script that runs your model on an input directory containing images and saves a CSV file named "results.csv" with the structure "*file_name*","*prediction*" for each row where "*file_name*" is the name of an input image and "*prediction*" is either "alcohol" or "non_alcohol". See the given example_results.csv file in the data directory for a formatting example.

2. Reports & Visualizations
    - Summarize your data exploration, methodology, and final performance metrics.
    - Show a few failure cases and discuss possible improvements.

3. Additional Notes
    - Provide any relevant instructions or scripts that help us run and evaluate your model.
    - Real-world classification often requires **iterative refinement**—we encourage you to share partial success and highlight where your pipeline might still fail, plus how you'd improve it with more time.
    - Expected time: ~10-15 hours total, balancing depth and clarity.
        - Feel free to reach out should you have any questions/need some clarification
        - This is not a Kaggle competition - It is advised to spend time on research and reasoning rather than trying to squeeze another 0.1% improvement in your metrics

Enjoy, and good luck!

©2020 DoubleVerify. All rights reserved.

Appendix - OCR minimal examples

EasyOCR

Install:

```
pip install easyocr
```

Use:

```
import easyocr
reader = easyocr.Reader(['en']) # initialize the model once
result = reader.readtext('some_image.jpg')
```

PaddleOCR

Install:

```
pip install paddlepaddle==3.0.0rc1 -i
https://www.paddlepaddle.org.cn/packages/stable/cpu/
pip install paddleocr
```

Use:

```
from paddleocr import PaddleOCR

reader = PaddleOCR(use_angle_cls=True, lang='en') # initialize the model
once
result = reader.ocr('some_image.jpg', cls=True)
```


©2020 DoubleVerify. All rights reserved.